

A Survey of Student Sentiments on AI-Assisted Software Engineering Courses

Skyler Oakeson
Utah State University
Logan, UT, USA
skyler.oakeson@usu.edu

Seth Poulsen
Utah State University
Logan, UT, USA
seth.poulsen@usu.edu

Abstract

This is a paper

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

Skyler Oakeson and Seth Poulsen. 2018. A Survey of Student Sentiments on AI-Assisted Software Engineering Courses. *ACM J. Data Sci.* 37, 4, Article 111 (August 2018), 2 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

In this document we discuss how to write an ACM article.

2 Related Work

Research on generative AI (GenAI) in programming and software engineering education has grown quickly, and can be organized into three broad strands that motivate the present survey: (1) how courses and curricula are being redesigned around AI-assisted development, (2) how students perceive, use, and feel about AI tools while learning to program, and (3) how these perceptions relate to measurable outcomes such as performance, self-efficacy, and skill development.

2.1 Redesigning Courses and Curricula for AI-Assisted Development

As generative AI coding tools have become embedded in professional software development, universities have begun to design dedicated courses and to rework existing ones around AI-assisted workflows. Geng et al. [2] analyzed the syllabi and course materials of 23 U.S. upper-division, credit-bearing courses that explicitly frame generative AI as part of software engineering, characterizing their learning objectives, assessment methods, topics, and the AI tools they document, in order to map how this emerging curricular area is currently being defined. Complementing this curriculum-level view, Böttcher et al. [1] argue that because students already

use generative AI tools regardless of instructor attitudes toward them, educators need to explicitly incorporate systematic and professional AI-tool use into teaching and learning, which requires rethinking constructive alignment—that is, the fit between learning objectives, teaching and learning activities, and assessments—for introductory software development courses. Related work in this space has examined how instructors integrate AI tools into programming and software engineering curricula more broadly, including concerns about maintaining core competencies such as debugging, algorithmic thinking, and independent problem-solving alongside AI-assisted instruction [4].

2.2 Student Perceptions and Patterns of AI Use in Programming Courses

A second strand of work surveys or otherwise elicits how students themselves perceive and use AI tools once they are integrated into programming instruction. Kohen-Vacs, Usher, and Jansen [4] surveyed undergraduate students on the usefulness of GenAI tools for coursework, their potential to enhance creativity, students' behavioral intention to continue using them, and associated concerns; students reported generally favorable, medium-to-high perceptions of usefulness and creativity support and strong intentions to keep using GenAI tools, while expressing comparatively modest concerns centered mainly on the risk of over-reliance. In a complementary study conducted within introductory programming labs, Güner and Er [3] identified five distinct profiles of student-AI interaction—ranging from AI-reliant code generation to more independent or collaborative styles—and showed that these profiles shifted substantially once students received explicit training in prompting and were given example prompts, with better-performing profiles becoming more common as guidance increased. This finding echoes a broader theme in the literature that unguided or unstructured use of GenAI tools can be less beneficial than guided, scaffolded use, a pattern also reported for structured prompt training in engineering and data-analysis courses.

Other work has probed student sentiment through more targeted or qualitative instruments. Pădurean et al. [6], studying more than 900 CS1 students attempting dialogue-based “Prompt Problems”—exercises in which students write natural-language prompts for a code-generating model rather than code itself—found that students generally rated prompting as easier and more enjoyable than traditional coding tasks and perceived it as well suited to developing problem-solving skills, while also cataloguing the specific prompt-level mistakes (chiefly omission of key details) that led to unsuccessful attempts. Thorgeirsson, Weidmann, and Su [7] took a related but distinct angle, running a controlled, preregistered study of “vibe coding”—programming purely through natural-language

Authors' Contact Information: Skyler Oakeson, Utah State University, Logan, UT, USA, skyler.oakeson@usu.edu; Seth Poulsen, Utah State University, Logan, UT, USA, seth.poulsen@usu.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2831-3194/2018/8-ART111
<https://doi.org/XXXXXXX.XXXXXXX>

prompting and observed behavior, without viewing the underlying code—and found that both written-communication skill and computer-science achievement independently and significantly predicted vibe-coding performance, with prompt quality mediating much of the association between writing skill and outcomes.

2.3 Linking Student Attitudes to Performance and Self-Efficacy

A third strand connects student sentiment more directly to measured outcomes. Prather et al. [5] developed and validated a self-efficacy scale for programming with generative AI, adapting the established Steinhurst self-efficacy subscales (covering tracing, patterns, problem solving, and control) to a GenAI-forward introductory computing context, and found that these foundational constructs remained reliable even when the surrounding curriculum incorporated AI tools throughout. Kohen-Vacs, Usher, and Jansen [4] additionally examined students' technical performance on a task requiring them to correct flawed, LLM-generated code during exams, finding that students performed significantly worse on these debugging tasks than on comparable instructor-designed programming and theoretical exam components—a gap that surfaced despite students' generally positive stated attitudes toward GenAI tools, underscoring a divergence between favorable sentiment and demonstrated competence in critically evaluating AI output. Taken together with the profile- and performance-linked findings of Güner and Er [3], this body of work suggests that student sentiment toward AI-assisted instruction cannot be assumed to track straightforwardly with learning gains, motivating survey instruments—like the one used in the present study—that probe sentiment as a construct distinct from, but related to, performance and self-efficacy.

2.4 Positioning of the Present Study

Existing research thus offers converging evidence that (a) AI-assisted software development courses are a newly forming and heterogeneous curricular category, (b) students hold generally favorable but multidimensional attitudes toward AI tools that vary with prior instruction, guidance, and prompting skill, and (c) these attitudes are only loosely coupled with objective performance on AI-related tasks. However, much of this work either surveys perceptions in a single course or intervention, or infers sentiment indirectly from interaction logs and performance data rather than through validated, retrospective self-report instruments aimed specifically at students who have completed a full AI-assisted software development course. The present study addresses this gap by directly surveying students who have been enrolled in such courses about their sentiments toward the experience, complementing the curriculum-mapping, interaction-profiling, and performance-linked perspectives established in prior work.

3 Methods

4 Results

5 Discussion

6 Conclusion

References

- [1] Axel Böttcher, Veronika Thurner, and Benedikt Zönnchen. 2025. Concepts for Teaching Software Development in the Age of AI-Tools. In *2025 IEEE Global Engineering Education Conference (EDUCON)*. doi:10.1109/EDUCON62633.2025.11016505
- [2] Francis Geng, Anshul Shah, Haolin Li, Nawab Mulla, Steven Swanson, Gerald Soosai Raj, Daniel Zingaro, and Leo Porter. 2026. Mapping the Emerging Curriculum for AI-Assisted Software Engineering via Syllabus Analysis. arXiv:2608.05898 [cs.SE]. <https://arxiv.org/abs/2608.05898>
- [3] Hacer Güner and Erkan Er. 2025. AI in the classroom: Exploring students' interaction with ChatGPT in programming learning. *Education and Information Technologies* 30, 9 (2025), 12681–12707. doi:10.1007/s10639-025-13337-7
- [4] Dan Kohen-Vacs, Maya Usher, and Marc Jansen. 2025. Integrating Generative AI into Programming Education: Student Perceptions and the Challenge of Correcting AI Errors. *International Journal of Artificial Intelligence in Education* 35, 5 (2025), 3166–3184. doi:10.1007/s40593-025-00496-4
- [5] James Prather, Lauren Margulieux, Yekaterina Kharitonova, Yonggao Yang, Brent N. Reeves, Paul Denny, Jamie Gorson Benario, Ernest D. V. Holmes, Erin Spaulding, Gweneth Barbre, Musa Blake, and Juho Leinonen. 2026. A Validated Scale Measuring Student Self-Efficacy for Programming with Generative AI. In *Proceedings of the 2026 ACM Conference on International Computing Education Research (ICER '26)*. doi:10.1145/3765964.3811645
- [6] Victor-Alexandru Pădurean, Kaitlin Riegel, Gweneth Barbre, Musa Blake, Paul Denny, Alkis Gotovos, Juho Leinonen, Stephen MacNeil, James Prather, and Adish Singla. 2026. Understanding Student Perceptions, Mistakes, and Debugging Approaches when Solving Natural Language Programming Tasks. In *Proceedings of the 2026 ACM Conference on International Computing Education Research (ICER '26)*. doi:10.1145/3765964.3811639
- [7] Sverrir Thorgeirsson, Theo B. Weidmann, and Zhendong Su. 2026. Computer Science Achievement and Writing Skills Predict Vibe Coding Proficiency. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. doi:10.1145/3772318.3791666